竞赛动态规划优化

目录

1 从朴素 DP 出发认识优化 1
2 单调队列优化滑动窗口转移 2
3 斜率优化把决策变成直线 3
4 李超线段树处理无序直线 5
5 四边形不等式与决策单调性 7
6 分治优化分层 DP 8
7 Knuth 优化区间 DP 9
8 优化方法对照与选择 11
9 WQS 二分消去次数维度 11
10 从树状数组到 CDQ 分治 13

---

1 从朴素 DP 出发认识优化

假设要把数组的前 i 个元素分成若干个非空连续段，每一段都有代价。定义 $f(i)$ 为处理完前 i 个元素的最小总代价。

如果最后一段是第 $j+1$ 到第 i 个元素，那么前 j 个元素必须已经处理好。这个方案的总代价就是：

$$\underbrace{f(j)}_{\text{前面的最优代价}} + \underbrace{C(j,i)}_{\text{最后一段的代价}}.$$

枚举所有可能的最后一个断点，便得到：

$$f(i) = \min_{0\le j<i}\{f(j) + C(j,i)\}.$$

这行式子中的术语可以这样理解：

术语或符号	具体含义
状态 $f(i)$	当前想求的一个子问题的答案
候选决策 j	假设最后一段从 $j+1$ 开始
转移	用已经求出的 $f(j)$ 尝试更新 $f(i)$
$C(j,i)$	闭区间 $[j+1,i]$ 的代价，也记作 $(j,i]$
$f(0)=0$	空前缀不花代价，使“整段从第 1 个元素开始”也能统一处理
最优决策 $opt(i)$	真正使转移式取得最小值的那个 j

注意 `f[i]` 存**答案数值**，`opt[i]` 存**选择了哪个下标**。答案变大或变小，与最优下标是否单调，是两件不同的事。

例如 $a=[1,2,1]$，每段代价为“段内元素和的平方”，则：

$$f(1)=1,\qquad f(2)=\min(3^2,1+2^2)=5.$$

计算 $f(3)$ 时的三种候选为：

最后断点 j	最后一段	候选总代价
0	$[1,3]$	$0+4^2=16$
1	$[2,3]$	$1+3^2=10$
2	$[3,3]$	$5+1^2=6$

因此 $f(3)=6$，$opt(3)=2$。后面的很多优化，就是在保证结果不变的前提下，避免每次重新把这张候选表全部算一遍。

通常可以粗略拆成：

$$\text{复杂度} = \text{状态数} \times \text{每个状态枚举的决策数} \times \text{一次转移的计算代价}.$$

例如把前 i 个元素分为恰好 t 段：

$$f_t(i) = \min_{t-1\le j<i}\{f_{t-1}(j) + C(j,i)\}.$$

其中 $C(j,i)$ 表示最后一段 $(j,i]$ 的代价。有 $O(Kn)$ 个状态，每个状态枚举 $O(n)$ 个断点；如果代价已经能 $O(1)$ 计算，总复杂度仍为 $O(Kn^2)$。

前缀和只优化了代价计算；要继续提速，需要减少枚举、批量维护候选，或者消掉一维状态。

下表的复杂度均以满足适用条件为前提；分治、Knuth 中的一次代价查询默认是 $O(1)$。

转移或问题特征	优先考虑	典型复杂度
$\min_j\{f(j)+B(j)\}+A(i)$，候选不断加入	维护前缀最值	$O(n)$
在左右端点都向右移动的窗口内取固定候选值最小值	单调队列	$O(n)$
决策带一个值域范围限制	树状数组或线段树	常见 $O(n\log n)$
$A(i)+\min_j\{m_jx_i+b_j\}$	斜率优化	单调条件合适时 $O(n)$
直线插入顺序、查询横坐标均无序	李超线段树	离散后 $O(n\log n)$
分层 DP，固定层的最优断点随位置单调	分治优化	$O(Kn\log n)$

转移或问题特征	优先考虑	典型复杂度
区间合并 DP，最优断点有双侧夹逼	Knuth 优化	$O(n^2)$
恰好选 K 次或分 K 段，答案关于次数有适当凸性	WQS 二分	$O(T(n)\log V)$
前驱满足多维偏序条件	扫描线、树状数组、CDQ	常见 $O(n\log n)$ 或 $O(n\log^2 n)$
对所有子集求和、最值或布尔聚合	SOS DP	$O(B2^B)$
只维护可达性，转移是批量平移	bitset	常见 $O(nW/\omega)$
固定线性转移重复非常多次	矩阵快速幂	朴素乘法 $O(d^3\log T)$

这里 $T(n)$ 是一次带惩罚 DP 的代价，V 是二分参数范围，B 是集合位数，W 是值域，ω 是机器字位数，d 是矩阵维度。

四边形不等式是一种性质；分治优化和 Knuth 优化是利用决策范围的算法；斜率优化是把候选变成直线来维护。它们之间有交集，但不能互相直接替换。

下面代码使用 GNU C++17。各模板按需复制，共用：

```
#include <bits/stdc++.h>
using namespace std;
#define int long long
using i128 = __int128;
const int inf = (1LL << 62);
```

代码风格与数论、组合数学板子统一：使用 `#define int long long`、`vector<int>` 和 `signed main()`。默认 DP 值及转移运算可放入 `long long`，有限最值的绝对值小于 `inf`；`i128` 只用于凸包判定中的交叉乘法。

`inf` 在最小化 DP 中表示“暂时不可达”，遇到这样的状态要跳过转移。

代码中 `pre` 表示上一层，`cur` 表示当前层；`[&]` 表示匿名函数可以使用外层变量；`auto&& self` 是递归匿名函数的写法，`self(self, ...)` 就相当于普通递归调用。还不熟悉这种语法时，可以先按照旁边的递归伪代码理解执行过程。

代码块默认是可复用的函数或结构体，不是每一块都包含 `main`。复制时带上本节公共头文件与类型定义；CDQ 章节还给出了对应的输入输出主函数。

2 单调队列优化滑动窗口转移

考虑：

$$f(i)=A(i)+\min_{L(i)\le j\le R(i)}\{f(j)+B(j)\}.$$

如果：

1. $L(i),R(i)$ 都不下降；
2. 候选加入时 $f(j)$ 已经算好；
3. 候选的比较值 $v_j=f(j)+B(j)$ 加入后不变；

就可以维护一个下标递增、值递增的双端队列。

新候选 q 比队尾旧候选 p 更晚过期，且 $v_q\le v_p$，那么 p 在未来任何合法窗口里都不可能优于 q，可以永久删除。这同时解释了“按值删队尾”和“按下标删过期队头”。

如果候选之间谁更好还依赖当前 i，例如比较 $m_jx_i+b_j$，就不能直接按一个固定值维护队列，通常需要斜率优化。

设 $f(0)=0$，$k\ge 1$：

$$f(i)=a_i+\min_{\max(0,i-k)\le j<i}f(j).$$

`a` 按 1 下标传入，`a[0]` 不使用。

```
vector<int> window_dp(const vector<int> &a, int k) {
    assert(k >= 1);
    int n = (int)a.size() - 1;
    vector<int> f(n + 1);
    deque<int> q;
    q.push_back(0);
    for (int i = 1; i <= n; i++) {
        while (!q.empty() && q.front() < i - k)
            q.pop_front();
        f[i] = f[q.front()] + a[i];
        while (!q.empty() && f[q.back()] >= f[i])
            q.pop_back();
        q.push_back(i);
    }
    return f;
}
```

每个下标只入队一次、出队至多一次，时间 $O(n)$，队列空间 $O(\min(n,k))$。数组 `f` 占 $O(n)$；若只需要答案，还能另外优化存储。

手算 $k=2$、$a=[3,-4,2,1]$，其中 $f(0)=0$。表中的队列写作“下标对应的 DP 值”，并记录每轮插入新状态之后的结果：

当前 i	查询时的合法前驱	得到的 $f(i)$	维护后的队列
1	$j=0$	$0+3=3$	$(0,0),(1,3)$
2	$j=0,1$	$0-4=-4$	$(2,-4)$
3	$j=1,2$	$-4+2=-2$	$(2,-4),(3,-2)$
4	$j=2,3$	$-4+1=-3$	$(2,-4),(4,-3)$

在 $i=2$ 时，新状态值为 -4，比队尾的 3 和 0 都小，而且下标更大、过期更晚，因此把二者从队尾删除。在 $i=4$ 时，状态 4 支配状态 3，但不能支配值更小的状态 2。

队列**不必保留窗口内的所有下标**，只需保留仍可能成为最优解的下标。查询前删除过期元素，查询后插入刚算出的新状态，这两个步骤承担不同的任务。

一类物品重量 w、价值 v、最多 c 个。固定重量模 w 的余数 r，把容量写为 $r+tw$：

$$\begin{aligned} new[r+tw] &= \max_{0\le q\le \min(c,t)} \{old[r+(t-q)w]+qv\} \\ &= tv+\max_{\max(0,t-c)\le p\le t} \{old[r+pw]-pv\}. \end{aligned}$$

最后一行已经是固定候选值的滑动窗口最大值。每种物品处理全部余数类的总长度为 $O(W)$，从而得到每类物品 $O(W)$ 的转移。

要区分上一层 `old` 和这一层 `new`，避免读取已经更新的本层状态而突破物品数量限制。

3 斜率优化把决策变成直线

把转移整理成：

$$f(i)=A(i)+\min_{j<i}\{m_jx_i+b_j\}.$$

已经计算好的状态 j 对应直线 $y=m_jx+b_j$。计算 $f(i)$ 就是在 $x=x_i$ 查询所有合法直线的最小值。这个变形是 Convex Hull Trick 的入口；李超线段树也可以处理相同的直线集合查询问题。直线查询模型参考

注意 b_j 可以包含 $f(j)$。这意味着直线往往只能在 $f(j)$ 算完后加入，不能为了方便排序就把所有候选提前插入。

后面说的“下包络”，就是在每个横坐标处取所有直线的最低值，这些最低点连成的边界。第一次学习先把它理解成“从很多条直线中查询最小值”即可；第 3.3 节的点凸包视角可以在理解代码后再看。

设 S_i 为前缀和：

$$f(i)=\min_{0\le j<i}\{f(j)+(S_i-S_j)^2+C\},\quad f(0)=0.$$

展开平方：

$$f(i)=S_i^2+C+\min_{j<i}\{-2S_jS_i+f(j)+S_j^2\}.$$

对应关系为：

项	对应值
查询横坐标 x_i	S_i
候选斜率 m_j	$-2S_j$
候选截距 b_j	$f(j)+S_j^2$
查询外的常数 $A(i)$	S_i^2+C

如果 $a_i\ge 0$，那么 S_i 不下降，插入的斜率 $-2S_j$ 不上升，恰好适合后面的单调队列凸包模板。

如果有负数，代数变形仍成立，但前缀和可能无序。此时应换成李超线段树等结构，而不是继续删查询队头。

用一个数字例子检查这次变形。取 $a=[1,2]$、$C=3$，则 $S=[0,1,3]$：

1. 初始只有状态 0，对应直线 $L_0(x)=0$。
2. 计算状态 1 时，在 $x=1$ 查询得到 0，所以 $f(1)=1^2+3+0=4$。
3. 状态 1 已经确定，才能加入直线 $L_1(x)=-2x+5$，其中截距 $5=f(1)+S_1^2$。
4. 计算状态 2 时，在 $x=3$ 比较两条线：$L_0(3)=0$，$L_1(3)=-1$。所以 $f(2)=3^2+3-1=11$。

回到原转移验证：一段到底的代价是 $3^2+3=12$，从状态 1 转移的代价是 $4+2^2+3=11$，答案一致。

这说明查询出的直线值可以为负，它只是转移式拆出来的一部分；最后还要加上 S_i^2+C，才能得到完整答案。

也可以把候选写为点：

$$P_j=(X_j,Y_j)=(S_j,f(j)+S_j^2).$$

查询的是 $Y_j-2S_iX_j$。固定查询斜率 $k=2S_i$，把直线 $Y=kX+b$ 从下方向上平移，第一次碰到候选点时的截距 b 就是最小值，所以只需维护这些点的下凸包。

设 $X_p < X_q$，决策 q 不差于 p 当且仅当：

$$Y_q - kX_q \le Y_p - kX_p \Leftrightarrow \frac{Y_q - Y_p}{X_q - X_p} \le k.$$

这就是“比较候选点连线斜率与当前查询斜率”的来源。点视角和直线视角等价，选一种推导清楚即可；不要混用两种视角的符号。

下面统一为：**求最小值，直线斜率按不升顺序加入，查询 x 按不降顺序出现。**

两条不同斜率的线 a, b 满足 $m_a > m_b$，它们的交点横坐标为：

$$t(a,b) = \frac{b_b - b_a}{m_a - m_b}.$$

当 x 向右走时，较小斜率的线逐渐更有优势。维护凸包后，最优直线只会从队头向队尾移动。

对于斜率严格递减的三条线 a, b, c，如果：

$$t(a,b) \ge t(b,c),$$

则 b 没有独占的最优区间，可以删除。因为两个分母都为正，等价于：

$$(b_b - b_a)(m_b - m_c) \ge (b_c - b_b)(m_a - m_b).$$

例如 $a(x) = 3x$、$b(x) = 2x + 4$、$c(x) = x + 5$，两个交点分别为 4 和 1。b 还没来得及优于 a，就已经被 c 压下去了，因此不需要保留。

```
struct Line {
    int m, b;
    int eval(int x) const {
        return m * x + b;
    }
};

// 仅在 a.m > b.m > c.m 时使用。
bool redundant(const Line &a, const Line &b, const Line &c) {
    return ((i128)b.b - a.b) * ((i128)b.m - c.m)
        >= ((i128)c.b - b.b) * ((i128)a.m - b.m);
}

struct MonotoneCHT {
    deque<Line> q;

    // 必须按 m 不升的顺序加入；求最小值。
    void add(Line cur) {
        while (!q.empty() && q.back().m == cur.m) {
            if (q.back().b <= cur.b) return;
            q.pop_back();
        }
        while (q.size() >= 2 &&
               redundant(q[q.size() - 2], q.back(), cur))
            q.pop_back();
        q.push_back(cur);
    }

    // 查询 x 必须不下降；调用前至少加入一条线。
    int query(int x) {
        assert(!q.empty());
        while (q.size() >= 2 && q[1].eval(x) <= q[0].eval(x))
            q.pop_front();
        return q.front().eval(x);
    }
};
```

相同斜率只保留较小截距。查询时直接比较函数值，不做浮点除法；删线判定的交叉乘法保留 `(i128)`，避免普通乘法溢出。

若有 n 次插入和 n 次查询，每条线最多被删除一次，**总时间为 $O(n)$，单次操作是均摊 $O(1)$**。

下面 `s[0]=0`，前缀和不下降。

```
vector<int> square_dp_cht(const vector<int> &s, int C) {
    int n = (int)s.size() - 1;
    vector<int> f(n + 1);
    MonotoneCHT hull;
    hull.add({0, 0}); // j = 0
    for (int i = 1; i <= n; i++) {
        f[i] = hull.query(s[i]) + s[i] * s[i] + C;
        hull.add({-2 * s[i], f[i] + s[i] * s[i]});
    }
    return f;
}
```

顺序是“用 $j < i$ 的候选查询 → 得到 $f(i)$ → 插入状态 i”。如果先插入再查询，就引入了不合法的 $j = i$ 自转移。

插入斜率	查询横坐标	处理方法
不升	不降	上面的单调队列，总计 $O(n)$
不升	任意	保留整个凸包，二分查询，总计 $O(n \log n)$
任意	任意	李超线段树，离散后总计 $O(n \log n)$
全部候选已知且查询都允许使用它们	任意	可先按斜率排序建凸包，再查询

“斜率不升、查询不降”是这里模板的具体方向，不代表任何两个单调方向都能原样套用。求最大值或换方向时，需要重新推导或做一致的符号变换。

查询横坐标无序时，下面的版本保留全部有效直线，不能沿用会 `pop_front` 的版本：

```
struct BinaryCHT {
    vector<Line> h;
    void add(Line cur) {
        while (!h.empty() && h.back().m == cur.m) {
            if (h.back().b <= cur.b) return;
            h.pop_back();
        }
        while (h.size() >= 2 &&
               redundant(h[h.size() - 2], h.back(), cur))
            h.pop_back();
        h.push_back(cur);
    }
    int query(int x) const {
        assert(!h.empty());
        int l = 0, r = (int)h.size() - 1;
        while (l < r) {
            int mid = l + (r - l) / 2;
            if (h[mid + 1].eval(x) <= h[mid].eval(x)) l = mid + 1;
            else r = mid;
        }
        return h[l].eval(x);
    }
};
```

凸包上的交点从左到右递增，因此固定 x 时，相邻两条线的优劣只会切换一次，可以二分。

1. 展开后是否真的只剩一个交叉项 $m_j x_i$？
2. m_j, b_j 在插入时是否已经确定？
3. 候选斜率和查询横坐标分别按什么方向变化？
4. 是否处理了相同斜率和相同查询横坐标？
5. 候选是否永久有效？如果候选会按时间过期，上面的普通凸包不能直接照搬。
6. 是否按题目数据范围选择了合适的整数类型？
7. 如果需要恢复方案或额外按次数打破平局，是否连同所有删线规则一起处理？这些模板默认只求最优值。

4 李超线段树处理无序直线

仍然是维护 $y = mx + b$ 的最小值，但插入斜率无序，或者无法方便地维护单调凸包时，李超线段树很实用。

它依赖的核心性质是两条不同直线最多相交一次。把查询横坐标建成线段树，每个结点保存一条线；插入新线时，让在当前区间中点更优的线留在结点，另一条线只可能在左边或右边的一侧继续有用。

查询某个 x 时，取根到对应叶子路径上所有直线的最小值。结点保存的线不要求在整个区间都最优，正确性由整条路径共同保证。

设当前区间为 $[l, r]$，中点为 mid：

1. 比较两条线在 x_{mid} 的值，把较小者留在当前结点。
2. 落败线在中点不占优。因为两线之差是一次函数，它不可能在中点两侧的端点同时严格更优。
3. 如果它在左端点更优，只递归左儿子；否则若它在右端点更优，只递归右儿子；否则整段丢弃。

每次插入只沿一条路径下降，因而是 $O(\log M)$，其中 M 是离散后的横坐标数量。
例如查询坐标为 $[0,1,2,3,4]$，当前线为 $A(x)=2x$，新线为 $B(x)=-x+4$：

横坐标	$A(x)$	$B(x)$	较小者
0	0	4	A
2	4	2	B
4	8	0	B

中点是 $x=2$，所以让 B 留在根结点。剩下的 A 在左端点仍然更好，继续递归到左子树。查询 $x=0$ 时，沿路径同时看见根上的 B 和左子树中的 A，取最小值得 0；查询 $x=4$ 时，根上的 B 已经给出正确答案 0。
线段树中的 l, r, mid 是**排序后坐标数组的下标**，实际比较必须使用 `xs[l]`、`xs[mid]`、`xs[r]`。离散后的相邻坐标不需要等距。
如果查询横坐标都来自输入，可以预先排序去重。**只离散横坐标，不提前插入依赖** DP **值的直线。**
下面沿用第 3 节的 `Line`，支持任意顺序插入直线和查询。只支持构造时给出的横坐标；每条线在整个查询域内有效。

```
struct LiChao {
    vector<int> xs;
    vector<Line> tr;
    vector<unsigned char> has;

    explicit LiChao(vector<int> x) : xs(move(x)) {
        sort(xs.begin(), xs.end());
        xs.erase(unique(xs.begin(), xs.end()), xs.end());
        assert(!xs.empty());
        tr.resize(4 * xs.size());
        has.assign(4 * xs.size(), 0);
    }

    void add(Line cur) {
        add(1, 0, (int)xs.size() - 1, cur);
    }

    void add(int p, int l, int r, Line cur) {
        if (!has[p]) {
            has[p] = 1;
            tr[p] = cur;
            return;
        }
        int mid = l + (r - l) / 2;
        if (cur.eval(xs[mid]) < tr[p].eval(xs[mid]))
            swap(cur, tr[p]);
        if (l == r) return;
        if (cur.eval(xs[l]) < tr[p].eval(xs[l]))
            add(p * 2, l, mid, cur);
        else if (cur.eval(xs[r]) < tr[p].eval(xs[r]))
            add(p * 2 + 1, mid + 1, r, cur);
    }

    int query(int x) const {
        int id = (int)(lower_bound(xs.begin(), xs.end(), x) - xs.begin());
        assert(id < (int)xs.size() && xs[id] == x);
        return query(1, 0, (int)xs.size() - 1, id);
    }

    int query(int p, int l, int r, int id) const {
        int ans = has[p] ? tr[p].eval(xs[id]) : inf;
        if (l == r) return ans;
        int mid = l + (r - l) / 2;
        if (id <= mid)
            return min(ans, query(p * 2, l, mid, id));
        return min(ans, query(p * 2 + 1, mid + 1, r, id));
    }
};
```

空间 $O(M)$，预处理 $O(M\log M)$，每次插入和查询 $O(\log M)$。若 $M=1$，单次操作为常数时间。
将 `square_dp_cht` 中的 `MonotoneCHT hull;` 改为 `LiChao hull(s);`，其余插入、查询方式不变，就能处理前缀和不单调的同一个 DP；原来的数值范围约定仍需满足。
如果横坐标未来才知道，可以在已知整数值域上动态开点，复杂度按值域宽度的对数计算。如果一条线只在一段横坐标范围有效，则需要“线段插入”版本，常见单次插入复杂度为 $O(\log^2 M)$。这些不是上面整条直线模板已经支持的操作。

5 四边形不等式与决策单调性

对最小化问题，这里使用以下形式：

$$C(a,c)+C(b,d)\le C(a,d)+C(b,c),\qquad a\le b\le c\le d.$$

它对应常见的 Monge 性质。各资料对“凸四边形不等式”“凹四边形不等式”的叫法可能不同，实战中应直接写出公式确认方向。

对于一般矩阵 M，Monge 条件是：

$$M[p,x]+M[q,y]\le M[p,y]+M[q,x],\qquad p<q,x<y,$$

这里四个矩阵项都必须合法。区间代价只在特定三角区域内有定义，所以需要额外的下标顺序限制。

固定两个候选 $p<q$ 和两个位置 $i<i'$，并保证这四个转移都合法。四边形不等式可以改写为：

$$C(q,i')-C(p,i')\le C(q,i)-C(p,i).$$

也就是随着位置向右，较大候选 q 相对于较小候选 p 的代价差不会上升。

因此，一旦 q 已经不差于 p，后面 p 就不会重新严格超过 q。这正是决策不会“走回头路”的原因。

例如两个候选在三个位置的转移总代价可以是：

候选	位置 i_1	位置 i_2	位置 i_3
较小下标 p	7	13	20
较大下标 q	9	10	14
q 的代价减去 p 的代价	2	-3	-6

一开始 p 更好，后来 q 更好。差值不增加，表示 q 一旦取得优势，就不会再失去对 p 的优势。注意这里限制的是**两个候选的相对差值**，不是要求每一行的 DP 代价递增或递减。

考虑：

$$f_t(i)=\min_{t-1\le j<i}\{f_{t-1}(j)+C(j,i)\}.$$

固定层 t，定义：

$$A_j(i)=f_{t-1}(j)+C(j,i).$$

向每个候选对应的一行加常数 $f_{t-1}(j)$，不会改变 Monge 不等式，因为常数在两边各出现一次。

约定 $opt_t(i)$ 是**所有最优候选中最大的下标**。设 $q=opt_t(i)$，对任意更小的合法候选 $p<q$，有：

$$A_q(i)-A_p(i)\le 0.$$

根据上面的差值单调性：

$$A_q(i')-A_p(i')\le A_q(i)-A_p(i)\le 0.$$

所以后面这些更小候选不可能严格优于 q；打平时也会选下标更大的 q。新出现的候选下标只会更大，因此：

$$opt_t(i)\le opt_t(i+1).$$

这就是分治优化需要的决策单调性。四边形不等式是常见的**充分条件**，不是必要条件；如果能用交换论证等方法直接证明最优决策单调，也能使用分治优化。

设 $a_i\ge 0$，S_i 是前缀和，且：

$$C(j,i)=(S_i-S_j)^2.$$

对 $p\le q\le x\le y$，令：

$$u=S_q-S_p,\quad v=S_x-S_q,\quad w=S_y-S_x.$$

因为元素非负，$u,v,w\ge 0$，从而：

$$\begin{aligned}&C(p,y)+C(q,x)-C(p,x)-C(q,y)\\&=(u+v+w)^2+v^2-(u+v)^2-(v+w)^2\\&=2uw\ge 0.\end{aligned}$$

这就证明了所需方向的四边形不等式。这里非负条件用于保证 $uw\ge 0$；加入负数后不能直接沿用这个结论。

这个例子同时具有代数结构和决策单调性，因此既可以用斜率优化，也可以用分治优化。

性质	具体含义	用途
查询单调	x_i 不下降	允许单调凸包删除查询队头
插入斜率单调	m_j 按固定方向变化	允许从凸包末尾维护候选
一侧决策单调	$opt_t(i)\le opt_t(i+1)$	分层 DP 的分治优化
双侧决策夹逼	$opt(l,r-1)\le opt(l,r)\le opt(l+1,r)$	Knuth 优化
区间包含单调	$W(b,c)\le W(a,d)$，其中 $[b,c]\subseteq[a,d]$	Knuth 常用充分条件的一部分

“数组单调”“DP 值单调”都不能自动替代这些性质。

6 分治优化分层 DP

仍然考虑恰好分为 K 个非空段：

$$f_t(i)=\min_{t-1\le j<i}\{f_{t-1}(j)+C(j,i)\}.$$

边界为 $f_0(0)=0$，其他 $f_0(i)=+\infty$。必须满足：

1. 计算第 t 层时，第 $t-1$ 层已经完整计算好；
2. 固定一层，最优候选下标随 i 不下降；
3. 代价查询足够快。下面的 $O(Kn\log n)$ 结论按每次 $O(1)$ 计算。

分治优化的标准复杂度与上述决策单调条件可对照 Divide and Conquer DP 文档。

用 `solve(l, r, optL, optR)` 表示：计算本层的 $[l,r]$，并已知这些位置的最优候选都在 $[optL,optR]$ 内。

先计算 mid，枚举合法候选，得到最优断点 p。根据单调性：

- 左半边的位置，最优断点不超过 p；
- 右半边的位置，最优断点不小于 p。

于是递归为：

```
solve(l, mid - 1, optL, p)
solve(mid + 1, r, p, optR)
```

这是“先算中点，再缩小两边决策范围”，不是把 DP 下标排序，也不是把所有位置的最优断点设成中点的最优断点。

手算一个具体过程。长度为 8 的数组全是 1，每段代价为段内和的平方，当前要算“恰好两段”的这一层。上一层已经知道 $f_1(j)=j^2$，所以：

$$f_2(i)=\min_{1\le j<i}\{j^2+(i-j)^2\}.$$

初始调用 `solve(2, 8, 1, 7)`，首先算中点 $i=5$：

断点 j	前一段代价 j^2	最后一段代价 $(5-j)^2$	总代价
1	1	16	17
2	4	9	13
3	9	4	13
4	16	1	17

按“相等时取较大断点”的规则，得到 $f_2(5)=13$、$opt_2(5)=3$。因此：

- 算位置 $2,3,4$ 时，只需要考虑断点 $[1,3]$；每个位置仍要裁掉不满足 $j<i$ 的部分。
- 算位置 $6,7,8$ 时，只需要考虑断点 $[3,7]$。

例如左子问题先算 $i=3$，只查 $j=1,2$，得到最优断点 2；右子问题先算 $i=7$，只查 $j=3,4,5,6$，得到最优断点 4。继续递归，就能求出所有位置。

这里 `l,r` 表示**要计算哪些状态**，`optL,optR` 表示**这些状态可能从哪些前驱转移**。二者虽然都是下标范围，却是不同的两条轴，混淆它们是分治模板最常见的错误之一。

`cost(j, i)` 表示 $(j,i]$ 的代价，返回 `int`。模板假设每个非空段都合法且代价有限，因此对 $t\le i\le n$ 一定存在有效转移。

```
template<class Cost>
int partition_dc(int n, int K, Cost cost) {
    if (K < 0 || K > n) return inf;
    vector<int> pre(n + 1, inf), cur(n + 1, inf);
    pre[0] = 0;

    for (int t = 1; t <= K; t++) {
        fill(cur.begin(), cur.end(), inf);
        auto solve = [&](auto&& self, int l, int r, int optL, int optR) -> void {
            if (l > r) return;
            int mid = l + (r - l) / 2;
            int best = inf;
            int bestJ = -1;
            int hi = min(optR, mid - 1);
            for (int j = optL; j <= hi; j++) {
                if (pre[j] == inf) continue;
                int val = pre[j] + cost(j, mid);
                if (val <= best) { // 相等时取较大的 j
                    best = val;
                    bestJ = j;
                }
            }
            assert(bestJ != -1);
            cur[mid] = best;
            self(self, l, mid - 1, optL, bestJ);
```

```
            self(self, mid + 1, r, bestJ, optR);
        };
        solve(solve, t, n, t - 1, n - 1);
        pre.swap(cur);
    }
    return pre[n];
}
```

平方区间和可以这样调用，s 为非负数组的前缀和：

```
auto cost = [&](int j, int i) -> int {
    int d = s[i] - s[j];
    return d * d;
};
int answer = partition_dc(n, K, cost);
```

如果题目还有最大段长、禁止区间等额外限制，必须重新检查合法候选集合、决策单调性和“每个状态必有候选”的约定。不能只把非法代价设成无穷大，就默认原证明仍成立。

分治递归树有 $O(\log n)$ 层。在递归树的同一深度，各结点的候选区间有序分割，只有边界可能重复，候选总长度是 $O(n)$。

因此计算一整层 DP 需要 $O(n \log n)$，共 K 层为 $O(Kn \log n)$。滚动数组空间 $O(n)$，递归栈 $O(\log n)$。

如果代价用动态双指针维护，必须另算移动指针的总次数；不能把一次 `cost` 调用当成天然的 $O(1)$。

对于：

$$f(i) = \min_{j<i}\{f(j) + C(j,i)\},$$

右边的 $f(j)$ 是**同一层的新状态**。直接先算中点时，左边有些 $f(j)$ 可能尚未求出，因此不能原样套上面的分层分治模板。

这种在线依赖需要别的安排，例如在合适的转移结构下用 CDQ 的“算左侧 → 左侧更新右侧 → 算右侧”，或者用单调队列、凸包等在线数据结构。

对于平方区间和：

$$f_t(i) = S_i^2 + \min_{j<i}\left\{-2S_jS_i + f_{t-1}(j) + S_j^2\right\}.$$

非负数组下，每一层都可以维护单调凸包，把 $O(n \log n)$ 进一步降为 $O(n)$，总计 $O(Kn)$。

这一层的直线截距包含 `pre[j]`，应从 $i = t$ 开始扫描，在查询 i 前加入合法的 $j = i - 1$，并跳过上一层不可达状态。不要照抄单层 DP 后误用 `cur[j]`。

7 Knuth 优化区间 DP

考虑区间合并：

$$f(l,r) = W(l,r) + \min_{l\le k<r}\{f(l,k) + f(k+1,r)\}, \quad f(i,i) = 0.$$

这里一次转移需要两个子区间，外层公共代价 $W(l,r)$ 不依赖分割点 k。朴素算法枚举区间长度、左端点和分割点，时间 $O(n^3)$。

令 $opt(l,r)$ 为所有最优分割点中最大的一个。如果能证明：

$$\boxed{opt(l,r-1) \le opt(l,r) \le opt(l+1,r)},$$

那么只需要在相邻区间的最优决策之间枚举。这比普通的一侧决策单调性更强。

在上述转移和边界下，经典的 Knuth–Yao 充分条件是：对任意 $a \le b \le c \le d$，同时满足

$$\boxed{W(a,c) + W(b,d) \le W(a,d) + W(b,c)}$$

和

$$\boxed{W(b,c) \le W(a,d)}.$$

第一条是四边形不等式，第二条是区间包含单调性。它们共同保证所需的最优分割点夹逼。可以核对 Knuth 优化的条件；相关一般性结果来自 Yao 的原始论文。

这是一组充分条件。如果能直接证明双侧夹逼，也可以优化；不能因为某个问题不符合这组常见条件，就断言它一定不能用 Knuth。

证明有两步：

1. 在这些条件下，递推得到的 $f(l,r)$ 也满足同方向的四边形不等式。
2. 利用 f 的四边形不等式，比较两个分割点的差值，推出双侧夹逼。

第一步是 Knuth–Yao 的关键引理，证明按区间长度做强归纳，将较大区间沿最优断点拆开，再用较短区间的归纳结论和 W 的两个条件；它不能省略为“W 满足，所以 f 显然满足”。

第二步可以直接理解。固定 l，比较 $p < q$，记：

$$T_k(l,r)=f(l,k)+f(k+1,r)+W(l,r).$$

只要两个断点在相邻区间里都合法，由 f 的四边形不等式：

$$f(p+1,r-1)+f(q+1,r)\le f(p+1,r)+f(q+1,r-1).$$

移项得到：

$$T_q(l,r)-T_p(l,r)\le T_q(l,r-1)-T_p(l,r-1).$$

所以右端点增加时，较大的断点相对不会变差，推出：

$$opt(l,r-1)\le opt(l,r).$$

类似地，对共同合法的 $p<q$ 使用：

$$f(l,p)+f(l+1,q)\le f(l,q)+f(l+1,p),$$

可知左端点增加时，较大断点相对也不会变差，推出：

$$opt(l,r)\le opt(l+1,r).$$

新增或消失的边界断点与上述方向一致。相等时始终取较大断点，保证最优解有多个时，`opt` 的定义与比较规则保持一致。

有 n 堆非负重量的石子，每次只能合并相邻两堆，代价是合并后的总重量，求全部合成一堆的最小总代价。

设 S_i 为前缀和，则：

$$W(l,r)=S_r-S_{l-1}.$$

它满足区间包含单调性，因为外层区间只是多包含了一些非负元素。

四边形不等式在这里更强，直接取等号：

$$W(a,c)+W(b,d)=W(a,d)+W(b,c).$$

因此可以使用 Knuth 优化，把 $O(n^3)$ 降至 $O(n^2)$。

以四堆石子 $[10,20,30,40]$ 为例，先计算较短区间：

区间	区间合并最小代价	最优分割点
$[1,2]$	30	1
$[2,3]$	50	2
$[3,4]$	70	3
$[1,3]$	90	2
$[2,4]$	140	3

例如 $f(1,3)=\min(0+50,30+0)+60=90$，选择断点 2；含义是先把前两堆合并，再把这个整体与第三堆合并。

现在算 $f(1,4)$，已知两个边界决策为 $opt(1,3)=2$、$opt(2,4)=3$，因此只枚举 $k=2,3$：

$$k=2:\quad 30+70+100=200,$$

$$k=3:\quad 90+0+100=190.$$

答案是 190。朴素算法还会算 $k=1$，得到 $0+140+100=240$；Knuth 用已经证明的决策范围直接跳过了它。

`cost(l, r)` 表示闭区间 $[l,r]$ 的 $W(l,r)$。模板假设已经满足上一节的充分条件，或已直接证明最优决策夹逼。

```
template<class Cost>
int knuth_merge(int n, Cost cost) {
    if (n == 0) return 0;
    vector<vector<int>> f(n + 2, vector<int>(n + 2, 0));
    vector<vector<int>> opt(n + 2, vector<int>(n + 2, 0));
    for (int i = 1; i <= n; i++) opt[i][i] = i;

    for (int len = 2; len <= n; len++) {
        for (int l = 1; l + len - 1 <= n; l++) {
            int r = l + len - 1;
            int lo = max(l, opt[l][r - 1]);
            int hi = min(r - 1, opt[l + 1][r]);
            assert(lo <= hi);
            f[l][r] = inf;
            int w = cost(l, r);
            for (int k = lo; k <= hi; k++) {
                int val = f[l][k] + f[k + 1][r] + w;
                if (val <= f[l][r]) { // 相等时取较大的 k
                    f[l][r] = val;
                    opt[l][r] = k;
                }
            }
        }
    }
    return f[1][n];
}
```

石子合并调用方式：

```
auto cost = [&](int l, int r) -> int {
    return s[r] - s[l - 1];
};
int answer = knuth_merge(n, cost);
```

`opt[i][i]=i` 是递推边界。长度为 2 时，必须把枚举右边界裁到 `r-1`，否则会出现空子区间或访问错误。
固定区间长度 L，令：

$$q_l = opt(l, l+L-2).$$

区间 $[l, l+L-1]$ 的候选数至多为：

$$q_{l+1} - q_l + 1.$$

对所有合法 l 求和会望远镜消去：

$$\sum_{l=1}^{n-L+1} (q_{l+1} - q_l + 1) = q_{n-L+2} - q_1 + n - L + 1 = O(n).$$

长度有 $O(n)$ 种，故总时间是 $O(n^2)$。不是每个状态只枚举常数个候选，而是同一条长度对角线上的总枚举量为 $O(n)$。
空间为 $O(n^2)$。`f` 和 `opt` 都是二维数组，使用前按题目的 n 估算内存。
把石子重量允许为负，取：

$$a = [-2, -1, -1, -3].$$

区间和仍然满足四边形恒等式，但不满足区间包含单调性。直接用原始三次方 DP 得到：

区间	最小代价	最优断点
$[1,3]$	-7	2
$[2,4]$	-9	2
$[1,4]$	-16	1

三个断点都是唯一最优。此时 $opt(1,3) = 2 > 1 = opt(1,4)$，双侧夹逼已经失效。机械套模板会漏掉真正的最优断点 1。
这个反例说明：**单有 W 的四边形不等式，不能直接保证上述区间合并 DP 可以使用 Knuth 优化。**

8 优化方法对照与选择

比较项	斜率优化	分治优化	Knuth 优化
常见转移	$A(i) + \min_j(m_j x_i + b_j)$	$\min_j(f_{t-1}(j) + C(j,i))$	$W(l,r) + \min_k(f(l,k) + f(k+1,r))$
真正利用的结构	直线下包络	固定层最优断点单调	相邻区间最优断点双侧夹逼
是否必须有四边形不等式	不必	不必，是常见充分条件	不必，双侧夹逼才是直接要求
典型计算顺序	查询并在线插入	上一层完成后，分治算本层	按区间长度递增
典型优化后代价	每层 $O(n)$ 或 $O(n \log n)$	每层 $O(n \log n)$	整张区间表 $O(n^2)$
最容易错的地方	方向、相同斜率、溢出	把同层依赖当上一层	漏掉区间包含单调性或合法断点边界

做题时先写完整转移，判断属于哪种依赖形式，再决定采用什么性质；不要仅凭题目里有“分段”“平方”“区间”就套模板。

9 WQS 二分消去次数维度

设 $F(k)$ 是恰好使用 k 次操作的最小原始代价，目标是 $F(K)$。直接做 DP 可能需要显式维护次数，产生一维 K 的复杂度。
给每次操作增加惩罚 λ，求：

$$G(\lambda) = \min_k \{F(k) + \lambda k\}.$$

如果一次惩罚 DP 能同时返回最优惩罚代价和使用次数，就可以通过调整 λ，把最优次数推向目标 K。这也叫 Aliens trick 或拉格朗日松弛思路。
对于分段问题，每增加一段就支付一次 λ：

$$g(i) = \min_{j<i} \{g(j) + C(j,i) + \lambda\}, \quad g(0) = 0.$$

原来的 $f_t(i)$ 变为 $g(i)$，但能否准确恢复恰好 K 段，还要检查下一节的条件。
先暂时把 DP 当作一个能求最优值的过程，看看惩罚有什么效果。假设原始答案为：

$$F(1)=17,\quad F(2)=10,\quad F(3)=7,\quad F(4)=6.$$

加上惩罚后，每一列表示“原代价加上 $\lambda\times$ 次数”：

惩罚 λ	用 1 次	用 2 次	用 3 次	用 4 次	选出的次数
0	17	10	7	6	4
1	18	12	10	10	3
2	19	14	13	14	3
3	20	16	16	18	2
8	25	26	31	38	1

这里代价相等时取更少次数。惩罚越大，使用次数越多的方案越吃亏。若目标为 $K=2$，在 $\lambda=3$ 时最小惩罚代价是 16，去掉两次惩罚得到 $16-3\times2=10$，回到原来的 $F(2)$。

二分的对象是**惩罚值 λ**，检查的是最优方案的**使用次数**；最后输出的仍是原问题的代价，不是加过惩罚的值。

取 $\lambda_1<\lambda_2$，对应最优次数 k_1,k_2。根据最优性：

$$F(k_1)+\lambda_1k_1\le F(k_2)+\lambda_1k_2,$$
$$F(k_2)+\lambda_2k_2\le F(k_1)+\lambda_2k_1.$$

相加消去 F：

$$(\lambda_2-\lambda_1)(k_2-k_1)\le0,$$

所以 $k_2\le k_1$。

这个次数单调性不要求 F 凸，但**只有次数单调性还不够保证能恢复 $F(K)$**。

对最小化形式，常见充分条件是 F 离散凸：

$$F(k-1)+F(k+1)\ge2F(k),$$

等价于差分 $F(k)-F(k-1)$ 不下降。几何上，点 $(k,F(k))$ 都位于下凸包上或其边界上，调节 λ 相当于用不同斜率的支撑线接触这些点。

对任意 λ 总有：

$$G(\lambda)-\lambda K\le F(K).$$

若存在一个惩罚值使 K 本身也是惩罚目标的最优次数，等号才成立。在标准的连续可行次数区间与离散凸性条件下，能找到这样的支撑惩罚。

反例：设 $F(1)=0,F(2)=100,F(3)=0$。次数仍然随惩罚增大而不增加，但无论怎么调 λ，次数 2 都不可能成为最优；中间的点在下凸包上方。此时凭次数二分后硬减 λK，无法得到 $F(2)=100$。

所以“题目要求恰好 K 个”只是考虑 WQS 的信号，不是正确性证明。

以下是用于理解和暴力核对的 $O(n^2)$ 版本，`cost(j, i)` 为 $(j,i]$ 的原始代价。相同惩罚代价时取**更少的段数**。

```
template<class Cost>
pair<int, int> penalty_dp(int n, int lambda, Cost cost) {
    vector<int> f(n + 1, inf);
    vector<int> cnt(n + 1, inf);
    f[0] = 0;
    cnt[0] = 0;
    for (int i = 1; i <= n; i++) {
        for (int j = 0; j < i; j++) {
            if (f[j] == inf) continue;
            int val = f[j] + cost(j, i) + lambda;
            int c = cnt[j] + 1;
            if (val < f[i] || (val == f[i] && c < cnt[i])) {
                f[i] = val;
                cnt[i] = c;
            }
        }
    }
    return {f[n], cnt[n]};
}
```

这段代码本身没有优化转移。真正提速需要用单调队列、凸包或其他结构把一次 `penalty_dp` 降到 $T(n)$，整体才可能成为 $O(T(n)\log V)$。

如果把它与斜率优化结合，必须让直线的比较、相同斜率处理、共点候选删除都遵守次数平局规则。前面的普通凸包模板只保证数值最优，不能直接声称它也正确维护最优段数。

下面的规则限定在这些条件下：可行次数为完整的 $1,2,\dots,n$，$F(k)$ 为整数并且离散凸，目标 $1\le K<n$，带惩罚 DP 相等时优先更少次数。

选择整数边界 $lo<hi$，保证 `count(lo) > K`、`count(hi) <= K`，二分最小的满足 `count(lambda) <= K` 的整数 λ_0。最后返回：

$$F(K)=G(\lambda_0)-\lambda_0K.$$

为什么即使返回的次数小于 K，这个公式仍成立？记 $\Delta_k = F(k) - F(k-1)$。离散凸性使 Δ_k 不下降，而“最优次数不超过 K”等价于：

$$\Delta_{K+1} + \lambda \geq 0.$$

因此第一个合法整数惩罚恰为 $\lambda_0 = -\Delta_{K+1}$，此时 K 也是惩罚目标的最优次数，只是平局规则可能返回更小的那个。

例如 $F(1) = 6, F(2) = 4, F(3) = 2$，目标为 $K = 2$。当 $\lambda = 2$ 时，三个惩罚代价全部为 8；“更少次数”的规则会返回 1，但次数 2 同样最优，因此仍可恢复 $F(2) = 8 - 2 \times 2 = 4$。这就是要证明支撑关系，而不能要求二分过程一定返回恰好 K 次的原因。

边界必须覆盖所需惩罚，不能默认 $\lambda \geq 0$；有些问题需要负惩罚。$K = n$ 时可以直接计算所有单段的代价，不套用上述“左边界次数大于 K”的约定。

如果代价是实数、可行次数有空洞，或者题目采用相反的平局规则，需要重新推导二分边界和答案恢复方法，不能只改一个不等号。

10 从树状数组到 CDQ 分治

有一列物品，第 i 件有属性 a_i 和收益 w_i。按原顺序选出一个子序列，并要求选出的 a 严格递增，求最大收益。

令 $f(i)$ 表示**必须以第 i 件结尾**的最大收益。可以只选第 i 件，也可以接在合法的前驱后面：

$$f(i) = w_i + \max\left(0, \max_{j<i,\, a_j<a_i} f(j)\right).$$

其中的 0 表示“从当前物品开始，前面不选”。即使收益允许负数，这个状态定义也成立。

最直接的写法是对每个 i 枚举全部 $j < i$，检查 $a_j < a_i$，总时间 $O(n^2)$。优化的入口是：**我们并不关心最好的前驱具体是谁，只关心属性小于 a_i 的前驱里，最大的 DP 值是多少。**

因此按 $i = 1, 2, \ldots, n$ 扫描，维护下面两个操作就够了：

- `update(pos, value)`：把属性位置 `pos` 上的候选值更新为最大值。
- `query(pos)`：查询属性位置 1 到 `pos` 之间的最大值。

树状数组可以让这两个操作各花 $O(\log n)$。原来的“枚举所有前驱”，就变成了“一次前缀最大值查询”。

假设属性值是 $[-100, 7, 10^9]$，直接按属性开数组显然太大。我们只需要比较大小，不需要保留数值间的距离，因此把它们排序、去重后映射为排名 $[1, 2, 3]$，这叫**离散化**。

相同原值必须映射到相同排名。设 a_i 的排名为 r_i，那么：

前驱条件	应当查询的前缀
$a_j < a_i$	`query(r_i - 1)`
$a_j \leq a_i$	`query(r_i)`

按原下标扫描保证 $j < i$，查询的前缀范围保证属性条件。计算完 $f(i)$ 后再 `update(r_i, f(i))`，避免自己给自己转移。

这里使用的是“不断加入候选，取最大值”的树状数组。如果需要把某个已存值任意降低，不能直接调用相同的 `update` 期待它撤销旧最大值。

现在每件物品有两个属性 a_i, b_i，要求子序列中两个属性都**不下降**。注意本节开始改成了非严格条件，后面会分别说明如何处理严格条件。

状态仍为以 i 结尾的最大收益：

$$f(i) = w_i + \max\left(0, \max_{\substack{j<i \\ a_j \leq a_i \\ b_j \leq b_i}} f(j)\right).$$

一条合法转移 $j \to i$ 必须同时满足三个条件：时间下标在前、a 不超过当前值、b 不超过当前值。

只用一棵按 b 建立的树状数组，查询结果里可能混入 $a_j > a_i$ 的非法候选。若直接把所有物品按 a 排序，则又可能让原本在后面的物品跑到前面，破坏 $j < i$。

CDQ 的做法是：保留原下标的先后关系，用递归选出一批“左边更新右边”的转移；在这批转移内部，时间条件天然成立，剩下的两个属性条件就能用排序和树状数组处理。

考虑一段原下标 $[l, r]$，取中点 mid。对这段内部满足 $j < i$ 的所有转移，只有三种可能：

类别	前驱 j 在哪里	当前状态 i 在哪里	由谁处理
左内部	$[l, mid]$	$[l, mid]$	左子问题递归
跨区间	$[l, mid]$	$[mid+1, r]$	当前结点批量处理
右内部	$[mid+1, r]$	$[mid+1, r]$	右子问题递归

不存在“右边更新左边”，因为那样 $j > i$，不合法。

这就是 CDQ 的分治对象：**把一大批转移按前驱与目标的位置拆开，让每个递归结点只负责从左半边到右半边的贡献。** 标准 CDQ 也用这种点对分类来组织计算；用于 DP 时，还需要特别安排递归和转移的先后顺序。CDQ 分治参考

对 $n = 4$，递归树是：

```
                [1,4]
          当前负责 {1,2} → {3,4}
             /              \
          [1,2]              [3,4]
        负责 1 → 2          负责 3 → 4
         /    \              /    \
       [1]    [2]          [3]    [4]
```

例如 $1 \to 4$ 在根结点处理，$1 \to 2$ 在左子结点处理，$3 \to 4$ 在右子结点处理。属性条件不满足时，后面的扫描和查询会把这条转移过滤掉。

这里的候选值是 $f(j)$，它本身还需要通过 DP 计算。因此不能把两边都当作互不相关的独立问题先算完。

正确顺序是：

```
cdq(l, r):
    如果 l == r, 返回
    mid = (l + r) / 2
    cdq(l, mid)                    // 左边的答案先变成最终值
    用左边的最终答案更新右边       // 把外部贡献送入右边
    cdq(mid + 1, r)                // 再计算右边内部的传播
```

这与归并排序常见的“左右都处理完，再合并”在执行顺序上不同。静态计数中，一个点的贡献可能从输入开始就固定；DP 中的 $f(j)$ 会随着前驱转移而变化，必须尊重依赖。

用最简单的例子看错误顺序：四件物品的 (a, b) 依次为 $(1,1), (2,2), (3,3), (4,4)$，每件收益为 1。正确 DP 是 $[1, 2, 3, 4]$。

如果先独立处理左右两半，得到 $[1, 2]$ 和 $[1, 2]$；随后根结点做跨区间更新，只会把右半边改成 $[3, 3]$。此时 $f(3)$ 从 1 增加到了 3，但 $3 \to 4$ 早已处理过，不会自动重新执行，于是漏掉正确答案 $f(4) = 4$。

当前只允许左半边的 j 更新右半边的 i，所以 $j < i$ 已经成立。剩下要保证：

$$a_j \le a_i, \qquad b_j \le b_i.$$

具体做法如下：

1. 分别建立左右两边的**临时下标列表**，按 a 从小到大排序。
2. 按 a_i 从小到大扫描右侧查询，维护一个左侧指针。
3. 把所有 $a_j \le a_i$ 的左侧状态加入树状数组，在 b_j 的排名处存入 $f(j)$。
4. 查询 b_i 排名以内的前缀最大值，用它更新 $f(i)$。

当 a_i 增大时，满足 $a_j \le a_i$ 的左侧候选集合只会扩大，因此指针只往前走，每个左侧状态最多加入一次。

三个条件现在分别由三种机制负责：

条件	保证方式
$j < i$	只让原下标的左半边更新右半边
$a_j \le a_i$	按 a 扫描，只有达标的左侧状态才加入结构
$b_j \le b_i$	对离散化后的 b_i 查询前缀最大值

临时列表可以排序，原数组的时间顺序不改变。当前批次完成后清空临时树状数组，再进入右侧递归，避免其他批次的候选残留。

使用下面的数据，仍然允许从任意物品开始，两个属性都要求不下降：

原下标 i	a_i	b_i	收益 w_i
1	1	3	5
2	2	1	4
3	2	3	6
4	3	4	3

先初始化 $f = [5, 4, 6, 3]$，表示每个状态都可以只选自己。

第一步，递归计算左半边 $[1, 2]$。 $1 \to 2$ 虽然满足 $a_1 \le a_2$，但 $b_1 = 3 > b_2 = 1$，因此不能转移。左半边最终为 $f(1) = 5, f(2) = 4$。

第二步，计算根结点的跨区间贡献。 查询物品 3 时，两个左侧候选都满足 $a_j \le 2$，加入树状数组；查询 $b \le 3$，返回最大值 5，所以 $f(3) = 6 + 5 = 11$。查询物品 4 时，前缀 $b \le 4$ 也返回 5，所以暂时得到 $f(4) = 3 + 5 = 8$。

第三步，递归计算右半边 $[3, 4]$。 此时 $f(3)$ 已经是 11，而且 $3 \to 4$ 的两个属性条件都成立，于是 $f(4)$ 更新为 $11 + 3 = 14$。

时刻	$f(1)$	$f(2)$	$f(3)$	$f(4)$
只选自己	5	4	6	3
左半边完成	5	4	6	3
根结点跨区间更新后	5	4	11	8
右半边完成	5	4	11	14

最终最优子序列是 $1,3,4$，收益为 $5+6+3=14$。根结点处理完后，右半边的值仍可能继续增大；它们要等右侧递归完成后才是最终值。

固定任意一对 $j<i$。从递归树的根向下看：只要二者还在同一半边，就一起进入对应的子问题；总会有唯一一个结点，使 j 落在左边、i 落在右边。

这对转移恰好归该结点的跨区间步骤负责。往上的结点里二者还没分开，往下的结点里二者不再同时出现，所以不会被分配到两个不同批次。

算法没有逐一枚举这些点对，而是用查询把一批候选的最大贡献一起算出来。对于每个批次，排序和前缀查询又恰好筛出满足两个属性限制的候选。

递归还保持一个依赖条件：进入 `cdq(l,r)` 时，原下标小于 l 的所有前驱对区间内状态的贡献已经处理；左递归完成后，左边状态全部确定，才能安全地把它们用作候选。由这个条件逐层归纳，可知返回时整个 $[l,r]$ 都已算对。

代码解决本章的“两属性不下降的最大收益子序列”，输入数组采用 1 下标，使用第 1 章的公共定义。

树状数组的 `lowbit(p)=p & -p` 表示最低位的 1 对应的数值。例如 6 的二进制为 `110`，所以 `lowbit(6)=2`。结点 `tree[p]` 保存排名区间 $[p-lowbit(p)+1,p]$ 内的最大值。

查询前缀 $[1,6]$ 时，先读 `tree[6]` 覆盖的 $[5,6]$，再读 `tree[4]` 覆盖的 $[1,4]$，取两者最大值。因此查询下标不断减去 `lowbit`。更新排名 3 时，依次访问结点 $3,4,8,\ldots$，把所有包含这个位置的相关结点更新，因此下标不断加上 `lowbit`。排名从 1 开始，查询排名 0 则直接返回基准值 0。

```
struct FenwickMaxCDQ {
    int n;
    vector<int> tree;
    vector<unsigned char> dirty;
    vector<int> touched;

    explicit FenwickMaxCDQ(int n_)
        : n(n_), tree(n_ + 1, 0), dirty(n_ + 1, 0) {}

    void update(int p, int value) {
        for (; p <= n; p += p & -p) {
            if (value <= tree[p]) continue;
            if (!dirty[p]) {
                dirty[p] = 1;
                touched.push_back(p);
            }
            tree[p] = value;
        }
    }

    int query(int p) const {
        int answer = 0; // 允许从当前物品开始，不接任何前驱
        for (; p > 0; p -= p & -p)
            answer = max(answer, tree[p]);
        return answer;
    }

    void clear_batch() {
        // 只清理本批次真正改过的树状数组结点。
        for (int p : touched) {
            tree[p] = 0;
            dirty[p] = 0;
        }
        touched.clear();
    }
};
```

```
vector<int> cdq_weighted_chain(const vector<int> &a,
                               const vector<int> &b,
                               const vector<int> &w) {
    assert(!a.empty() && a.size() == b.size() && b.size() == w.size());
    int n = (int)a.size() - 1;
    vector<int> f(n + 1, 0);
    if (n == 0) return f;

    // 只离散化 b；a 可以直接用原值比较。
    vector<int> values(b.begin() + 1, b.end());
    sort(values.begin(), values.end());
    values.erase(unique(values.begin(), values.end()), values.end());
    vector<int> rank_b(n + 1);
    for (int i = 1; i <= n; i++) {
        rank_b[i] = (int)(lower_bound(values.begin(), values.end(), b[i])
                          - values.begin()) + 1;
        f[i] = w[i]; // 初始方案：只选择自己
    }
    FenwickMaxCDQ bit((int)values.size());

    auto cdq = [&](auto&& self, int l, int r) -> void {
        if (l == r) return;
        int mid = l + (r - l) / 2;

        // 1. 先让左侧状态成为最终值。
        self(self, l, mid);

        // 2. 准备当前这一批“左侧更新右侧”。
        vector<int> left, right;
        for (int i = l; i <= mid; i++) left.push_back(i);
        for (int i = mid + 1; i <= r; i++) right.push_back(i);
        auto by_a = [&](int x, int y) {
            if (a[x] != a[y]) return a[x] < a[y];
            return x < y;
        };
        sort(left.begin(), left.end(), by_a);
        sort(right.begin(), right.end(), by_a);

        int p = 0;
        for (int i : right) {
            // 只加入 a_j <= a_i 的左侧候选。
            while (p < (int)left.size() && a[left[p]] <= a[i]) {
                int j = left[p++];
                bit.update(rank_b[j], f[j]);
            }
            // 前缀范围再保证 b_j <= b_i。
            int best_predecessor = bit.query(rank_b[i]);
            f[i] = max(f[i], w[i] + best_predecessor);
        }
        bit.clear_batch();

        // 3. 右侧收到左侧贡献之后，才能继续内部递推。
        self(self, mid + 1, r);
    };
    cdq(cdq, 1, n);
    return f;
}
```

树状数组中的 0 是“可以不接前驱”的基准，不是所有最大化 DP 通用的无穷小。负的前驱收益不会优于直接从当前物品开始，因此这里可以忽略它们；但最终的 $f(i)$ 仍可能为负。

如果要求必须从指定起点开始，就需要按题意把不可达状态设成负无穷，跳过不可达候选，同时修改树状数组的初值。不能继续套用“每个位置都能从自己开始”的初始化。

把下面的 `main` 与公共定义、上面的树状数组结构和 CDQ 函数一起使用即可运行。输入第一行为 $n \geq 1$，接下来每行依次是 a_i, b_i, w_i，输出一个**非空**合法子序列的最大收益。

```
signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    if (!(cin >> n)) return 0;
    assert(n >= 1);
    vector<int> a(n + 1), b(n + 1), w(n + 1);
    for (int i = 1; i <= n; i++) cin >> a[i] >> b[i] >> w[i];
    auto f = cdq_weighted_chain(a, b, w);
    int answer = *max_element(f.begin() + 1, f.end());
    cout << answer << '\n';
}
```

样例输入：

```
4
1 3 5
2 1 4
2 3 6
3 4 3
```

样例输出：

```
14
```

`f[n]` 只代表“必须以最后一件结尾”的答案。题目允许在任意位置结束，所以要取所有 `f[1..n]` 的最大值。若允许整个子序列为空，再与 0 取最大值；若不允许为空，全负权时答案就应该是负数。

大小为 s 的递归区间需要排序 $O(s\log s)$，并进行至多 $O(s)$ 次树状数组操作，每次 $O(\log n)$。在递归树的同一深度，所有区间长度之和为 n，因此这一层花 $O(n\log n)$。

递归深度为 $O(\log n)$，总时间为 $O(n\log^2 n)$。上述实现的峰值辅助空间为 $O(n)$：一个全局树状数组，加上当前递归路径上总长度为 $O(n)$ 的临时下标列表。

为什么不在每个递归结点直接 `fill(tree.begin(), tree.end(), 0)`？递归结点有 $O(n)$ 个，每次都清空大小为 n 的数组，会引入 $O(n^2)$ 开销。代码用 `touched` 记录本批次真正修改的位置，清理量就能被本批次的更新操作数控制。

这里每一批开始时树状数组都为空，所以可以把改过的位置直接还原成 0。这不同于带有上一层有效数据的嵌套数据结构；后者通常要保存旧值并回滚，不能一律清零。

情况	应当怎样处理
要求 $a_j < a_i$	把加入候选的 `while` 条件从 `<=` 改为 `<`
要求 $b_j < b_i$	查询 `rank_b[i]-1`，更新位置仍为 `rank_b[j]`
两个属性都相等	本章的非严格模型允许按时间先后继续转移，不能随意去重物品
想按 a 排序原数组	只排序临时下标列表，原时间下标必须保留
$f(i)$ 被多个递归批次更新	使用 `max(f[i], w[i]+候选值)`，不能覆盖已有更优值
把更新写成 `f[i] += 候选值`	会反复累加同一件物品的收益，和转移含义不符
右递归放在跨区间更新前	会漏掉新答案继续向右传播产生的贡献
树状数组未清理	可能混入不属于当前左侧的候选，甚至出现未来状态给过去转移

CDQ 不要求最优决策单调，也不要求四边形不等式。它要求的是：依赖能按某个先后维度组织，并且“左边对右边的一批转移”能比逐对枚举更快地算出。

本章用排序加树状数组完成批量转移；在别的问题里，这一步也可能用凸包、线段树或其他结构。先把当前这一种模型写对，再尝试替换其中的数据结构。